

spring security intrevwe qiestion asked in java intervwers
with detail answer

Here's a **comprehensive list of Spring Security interview questions and detailed answers** — frequently asked in **Java developer interviews** (especially for Spring Boot backend roles).



Spring Security Interview Questions and Answers

1. What is Spring Security?

Answer:

Spring Security is a powerful and customizable **authentication and authorization framework** for Java applications.

It provides protection against common attacks such as:

- Session fixation
- Clickjacking
- Cross-site request forgery (CSRF)
- Cross-site scripting (XSS)

It integrates seamlessly with Spring Boot and supports:

- Form-based login
- Basic and Digest authentication
- OAuth2 and JWT
- Method-level security

2. What is the difference between Authentication and Authorization?

Answer:

Aspect	Authentication	Authorization
Purpose	Verifies <i>who you are</i>	Verifies <i>what you can do</i>

Aspect	Authentication	Authorization
Example	Logging in with username/password	Checking access to an admin page
Provided By	AuthenticationManager	AccessDecisionManager OR @PreAuthorize

3. How does Spring Security work internally?

Answer:

When a user sends a request to a secured endpoint:

1. **Security Filter Chain** intercepts the request.
2. The **UsernamePasswordAuthenticationFilter** extracts credentials.
3. The **AuthenticationManager** delegates to **AuthenticationProvider**.
4. The **UserDetailsService** loads user info from DB.
5. If valid → an **Authentication object** is stored in `SecurityContextHolder`.
6. The request proceeds to the controller.
7. On logout or session end, `SecurityContext` is cleared.

4. What is the `SecurityFilterChain` in Spring Security?

Answer:

It's a chain of servlet filters that apply security logic before a request reaches the controller.

In modern Spring Boot (5.7+), we configure it using a bean:

```
java
```

 Copy code

```
@Bean
```

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exceptio
    http
        .csrf().disable()
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin/**").hasRole("ADMIN")
```

```

        .requestMatchers("/user/**").authenticated()
        .anyRequest().permitAll()
    )
    .formLogin(Customizer.withDefaults());
return http.build();
}

```

5. What is UserDetailsService ?

Answer:

It's an interface used to retrieve user-related data (like username, password, roles) from a custom source (e.g., DB).

Example implementation:

java

 Copy code

```

@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepo;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = userRepo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));
        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            getAuthorities(user)
        );
    }

    private Collection<? extends GrantedAuthority> getAuthorities(User user) {
        return user.getRoles().stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role.getName()))
            .collect(Collectors.toList());
    }
}

```

```
}  
}
```

6. What is PasswordEncoder and why do we use it?

Answer:

PasswordEncoder is used to **hash passwords** before storing them in the database. It helps prevent plaintext password storage.

Example:

```
java
```

 Copy code

```
@Bean  
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

When saving a user:

```
java
```

 Copy code

```
user.setPassword(passwordEncoder.encode(rawPassword));
```

7. How do you secure REST APIs with JWT in Spring Security?

Answer:

Steps:

1. **Authenticate** user with `/login` endpoint.
2. Generate a **JWT token** upon successful authentication.
3. Client includes JWT in `Authorization: Bearer <token>` header.
4. Create a custom **JWT filter** that:
 - Parses the token
 - Validates it
 - Sets authentication in `SecurityContextHolder`

5. Secure endpoints via role-based access.

8. What is CSRF and how does Spring Security handle it?

Answer:

CSRF (Cross-Site Request Forgery) is an attack where an unauthorized command is transmitted from a user that the web app trusts.

Spring Security automatically enables **CSRF protection** for web apps with form login.

For REST APIs (stateless), it's usually disabled:

java

 Copy code

```
http.csrf().disable();
```

9. What are Roles and Authorities in Spring Security?

Answer:

- **Authority:** A permission, like "READ_PRIVILEGE" or "WRITE_PRIVILEGE" .
- **Role:** A group of authorities. Spring prepends "ROLE_" by default.

Example:

java

 Copy code

```
.hasRole("ADMIN") // ROLE_ADMIN  
.hasAuthority("READ_PRIVILEGE")
```

10. How to apply method-level security?

Answer:

Enable it:

java

 Copy code

`@EnableMethodSecurity`

Use annotations:

java

 Copy code

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }
```

Other annotations:

- `@Secured("ROLE_ADMIN")`
- `@PostAuthorize("returnObject.owner == authentication.name")`

11. What is the difference between `@PreAuthorize` and `@Secured` ?

Annotation	Expression Support	Example
<code>@Secured</code>	No SpEL support	<code>@Secured("ROLE_ADMIN")</code>
<code>@PreAuthorize</code>	Supports SpEL	<code>@PreAuthorize("hasRole('ADMIN') and #id == authentication.principal.id")</code>

12. What is `AuthenticationManager` ?

Answer:

It's the core interface that **authenticates user credentials**.

It delegates authentication to one or more `AuthenticationProvider` s.

Example:

java

 Copy code

```
@Autowired
AuthenticationManager authManager;

Authentication auth = authManager.authenticate(
```

```
new UsernamePasswordAuthenticationToken(username, password)
);
```

13. How to customize login and logout in Spring Security?

java

 Copy code

```
http.formLogin()
    .loginPage("/login")
    .defaultSuccessUrl("/home")
    .failureUrl("/login?error=true");

http.logout()
    .logoutUrl("/logout")
    .logoutSuccessUrl("/login?logout=true");
```

14. How to permit public endpoints and secure others?

java

 Copy code

```
http.authorizeHttpRequests()
    .requestMatchers("/public/**", "/login", "/register").permitAll()
    .anyRequest().authenticated();
```

15. What are some common Spring Security annotations?

Annotation	Purpose
@EnableWebSecurity	Enables Spring Security configuration
@EnableMethodSecurity	Enables method-level security
@PreAuthorize	Checks access before method call
@Secured	Checks access by roles

Annotation	Purpose
@WithMockUser	Used in tests to simulate logged-in user

Would you like me to provide a **set of 10 advanced Spring Security interview questions** (e.g., JWT filters, OAuth2, stateless vs stateful sessions, SecurityContext propagation, etc.) next?



filter vs interceptor

Great question — understanding the **difference between Filter and Interceptor** is **very common in Spring interviews**, especially for backend developers.

Let’s break it down clearly 📌

⚙️ **Filter vs Interceptor in Spring**

Aspect	Filter	Interceptor
Package	javax.servlet (Servlet API)	org.springframework.web.servlet (Spring MVC)
Level	Works at the Servlet level (before Spring MVC)	Works at the Spring MVC level (after DispatcherServlet)
Execution Point	Before and after a request enters the DispatcherServlet	Before and after controller methods execution
Use Case	Used for low-level request processing — logging, authentication, encoding, CORS, etc.	Used for application-specific logic — authorization, modifying model/view, auditing

Aspect	Filter	Interceptor
Applies To	All incoming requests (including static resources)	Only requests handled by controllers (not static files)
Access to	Only the <code>HttpServletRequest</code> and <code>HttpServletResponse</code>	Has access to <code>Handler</code> (controller method) and <code>ModelAndView</code>
Configuration	Declared in <code>web.xml</code> or via <code>@Component</code> + <code>FilterRegistrationBean</code>	Declared by implementing <code>HandlerInterceptor</code> and registering via <code>WebMvcConfigurer</code>
Execution Order	Executed before Interceptors	Executed after Filters
Spring Bean Awareness	Not Spring-aware by default	Spring-managed bean (can use dependency injection)

Filter Example

java

 Copy code

```

@Component
public class LoggingFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        System.out.println("Filter: Request URI -> " + req.getRequestURI());
        chain.doFilter(request, response); // Continue chain
    }
}

```

→ Filters run **before** requests reach Spring MVC's `DispatcherServlet`.

Interceptor Example

java

```
@Component
public class AuthInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                            HttpServletResponse response,
                            Object handler) throws Exception {
        System.out.println("Interceptor: Before Controller -> " + request.getRequestURL());
        return true; // continue to controller
    }

    @Override
    public void postHandle(HttpServletRequest request,
                          HttpServletResponse response,
                          Object handler,
                          ModelAndView modelAndView) throws Exception {
        System.out.println("Interceptor: After Controller Execution");
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
                               HttpServletResponse response,
                               Object handler,
                               Exception ex) throws Exception {
        System.out.println("Interceptor: After View Rendered");
    }
}
```

Register it:

java

 Copy code

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Autowired
    private AuthInterceptor authInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
```

```
registry.addInterceptor(authInterceptor)
    .addPathPatterns("/api/**"); // apply only to specific paths
}
```

Key Summary

- **Filter** → Servlet level, works before Spring MVC.
 - **Interceptor** → Spring MVC level, works before/after controller methods.
 - **Order** → Filter → DispatcherServlet → Interceptor → Controller.
 - **Common Pattern:**
 - Use **Filter** for authentication/token extraction.
 - Use **Interceptor** for authorization or request tracking.
-

Analogy

Think of a web request like entering a building:

- **Filter** = Security guard at the **main gate** (checks everyone entering).
- **Interceptor** = Security inside the **office area** (checks employee access to certain